Requirements, Design and Tasks for rxnet

Python Implementation — Reactive execution networks

2026-05-20

Contents

Requirements Document: rxnet — Python Implementation **5**
Introduction . . . 5
Glossary . . . 5
Requirements . . . 6
Requirement 1: Shared Synchronous Phase Model . . . 6
Requirement 2: Context and Input Snapshot Handling . . . 6
Requirement 3: Deferred Action Execution . . . 7
Requirement 4: Generic Node Runtime API . . . 7
Requirement 5: FSM Semantics . . . 7
Requirement 6: Petri Net Semantics . . . 8
Requirement 7: PN Data Validation and Error Behavior . . . 8
Requirement 8: Python Frontend API Surface . . . 8
Requirement 9: Integration Patterns and Examples . . . 9
Requirement 10: Execution Model Constraints . . . 9

Design Document: rxnet — Python Implementation **11**
Overview . . . 11
Architecture . . . 12
High-Level Architecture . . . 12
Component Architecture . . . 12
Components and Interfaces . . . 13
Core Runtime . . . 13
Context . . . 13
FSM Frontend . . . 14
Petri Net Frontend . . . 14
Package API Surface . . . 15
Host Integration Layer . . . 15
Data Models . . . 15
Runtime Model . . . 15
FSM Model . . . 16
PN Model . . . 16

Correctness Properties . . . 16
Property 1: Phase Ordering Determinism . . . 17
Property 2: Snapshot Consistency Within Tick . . . 17
Property 3: Deferred Action Isolation . . . 17
Property 4: Deferred Queue Reset . . . 17
Property 5: Runtime Node Contract Safety . . . 17
Property 6: FSM First-Match Transition Rule . . . 17
Property 7: FSM No-Match Stability . . . 17
Property 8: FSM Deferred Action Behavior . . . 18
Property 9: PN Transition Enablement . . . 18
Property 10: PN Guard Enforcement . . . 18
Property 11: PN Commit Delta Correctness . . . 18
Property 12: PN Invalid Arc Exceptions . . . 18
Property 13: Python API Surface Stability . . . 18
Property 14: Example Executability . . . 18
Property 15: Host-Controlled Scheduling . . . 19
Error Handling . . . 19
Error Categories . . . 19
Error Handling Strategy . . . 19
Testing Strategy . . . 19
Unit Tests . . . 19
Property Tests . . . 19
Integration Tests . . . 20
Code Quality and Development Standards . . . 20
Non-Goals and Out-of-Scope . . . 20

Implementation Plan: rxnet — Python Implementation **21**
Overview . . . 21
Tasks . . . 21
Notes . . . 24

List of Figures

1 High-level architecture of the `rxnet` Python runtime and model frontends 12

Requirements Document: rxnet — Python Implementation

Introduction

This document specifies the requirements for the Python implementation of `rxnet`, a reactive synchronous runtime library that supports two model families: Finite State Machines (FSM) and Petri Nets (PN). The Python implementation provides a shared phase-based execution core and language frontend using dataclasses and standard-library modules.

The goal of this document is to capture what the Python library provides today, so future changes can be validated and reflected here first.

Glossary

- **RXNET_System**: The complete `rxnet` Python library surface
- **Core_Runtime**: Shared synchronous execution engine with phase-based ticking
- **Context**: Execution context containing live inputs, latched inputs, and deferred actions
- **Node**: Runtime unit with `evaluate` and `commit` phases
- **Deferred_Action**: Action scheduled during commit and executed after all commits
- **Activation_Group**: Set of nodes activated at the same logical instant
- **WCET**: Maximum measured complete node tick time, from latch start to dump end
- **FSM_Runtime**: FSM frontend wrapper over the core runtime (`rxnet.fsm.Runtime`)
- **FSM_Machine**: State machine node with transitions, guards, and optional actions (`rxnet.fsm.Machine`)
- **FSM_Transition**: Rule from `from_state` to `to_state`, with optional guard/action (`rxnet.fsm.Transition`)
- **PN_Runtime**: Petri Net frontend wrapper over the core runtime (`rxnet.pn.Runtime`)
- **PN_Net**: Petri net node with places, transitions (`rxnet.pn.Net`)
- **PN_Transition**: Petri transition with consume/produce arcs, optional guard/action (`rxnet.pn.Transition`)

- **Arc**: Petri arc with `place_id` and `weight` (`rxnet.pn.Arc`)
- **Tick**: One complete synchronous cycle: latch -> evaluate -> commit -> dump, with deferred action dispatch between commit and dump

Requirements

Requirement 1: Shared Synchronous Phase Model

User Story: As a model developer, I want deterministic phase-based execution, so that all nodes observe a consistent input snapshot and state updates happen predictably.

Acceptance Criteria

1. WHEN a tick starts, THE Core_Runtime SHALL latch input data before node evaluation
2. WHEN latching completes, THE Core_Runtime SHALL evaluate all registered nodes
3. WHEN evaluation completes, THE Core_Runtime SHALL commit all registered nodes
4. WHEN commit completes, THE Core_Runtime SHALL execute deferred actions and clear the queue before dump
5. WHEN deferred action dispatch completes, THE Core_Runtime SHALL dump outputs for all nodes
6. THE Tick order SHALL remain `latch -> evaluate -> commit -> dump` in the Python implementation, with deferred action dispatch between commit and dump
7. WHEN an executor activates multiple nodes at one instant, THE Context SHALL expose the same `activation_us` value to every node in that activation group

Requirement 2: Context and Input Snapshot Handling

User Story: As an integrator, I want separate live and latched inputs, so that guards read a stable snapshot during each tick.

Acceptance Criteria

1. THE Context SHALL expose mutable live inputs for application/driver updates
2. THE Context SHALL expose latched inputs used during evaluation
3. WHEN ticking, THE runtime SHALL update `latched_inputs` using `copy.copy(inputs)`

Requirement 3: Deferred Action Execution

User Story: As a runtime user, I want actions to run after all commits, so that side effects are deferred and model state commits are consistent.

Acceptance Criteria

1. WHEN a node commits with a proposed action, THE Context SHALL enqueue the action for deferred execution
2. THE Core_Runtime SHALL run deferred actions only after all node commits in the tick
3. THE deferred action queue SHALL be cleared after execution

Requirement 4: Generic Node Runtime API

User Story: As a library user, I want a generic runtime abstraction, so that different model families can share one execution engine.

Acceptance Criteria

1. THE Node contract SHALL include `evaluate` and `commit` callbacks
2. THE runtime SHALL support dynamic node list growth without explicit capacity configuration

Requirement 5: FSM Semantics

User Story: As an FSM author, I want first-match transition evaluation with deferred actions, so that machine behavior is deterministic and clear.

Acceptance Criteria

1. WHEN an FSM machine evaluates, THE machine SHALL reset `next_state` to current state and clear proposed action
2. THE FSM evaluator SHALL scan transitions in declaration order and select the first transition whose `from_state` matches and whose guard is true (or absent)
3. WHEN no transition matches, THE machine SHALL remain in the current state
4. WHEN committing, THE machine SHALL update current state to `next_state`
5. WHEN a transition action exists, THE action SHALL be enqueued as a deferred action (not executed inline during evaluate/commit)

Requirement 6: Petri Net Semantics

User Story: As a Petri Net author, I want place/transition execution with guards and arc validation, so that token flow is modeled consistently.

Acceptance Criteria

1. WHEN a PN net evaluates, THE net SHALL copy `places` into `next_places` and reset transition fire flags
2. THE PN evaluator SHALL mark a transition as fireable only if consume/produce arcs are valid and consume tokens are available
3. WHEN a transition guard exists, THE transition SHALL fire only if the guard returns true
4. WHEN committing, THE net SHALL apply consume/produce deltas for each fire-flagged transition
5. WHEN a transition action exists, THE action SHALL be enqueued as a deferred action

Requirement 7: PN Data Validation and Error Behavior

User Story: As a developer, I want invalid Petri model data to be detected, so that configuration errors fail early.

Acceptance Criteria

1. WHEN evaluating a PN net with an invalid arc `place_id`, THE runtime SHALL raise `IndexError`
2. WHEN evaluating a PN net with a negative arc weight, THE runtime SHALL raise `ValueError`

Requirement 8: Python Frontend API Surface

User Story: As a Python developer, I want ergonomic dataclass-based APIs, so that models are easy to define and run.

Acceptance Criteria

1. THE FSM frontend SHALL expose `Machine` and `Transition` dataclasses with optional guard/action callbacks
2. THE PN frontend SHALL expose `Net`, `Transition`, and `Arc` dataclasses

3. THE runtime wrappers (`rxnet.fsm.Runtime`, `rxnet.pn.Runtime`) SHALL expose `context`, model registration, and `tick()`
4. THE root package SHALL expose `fsm`, `pn`, `runtime`, and FSM-oriented aliases currently defined in `rxnet.__init__`
5. THE implementation SHALL rely only on standard-library modules present in the codebase (`dataclasses`, `typing`, `copy`)

Requirement 9: Integration Patterns and Examples

User Story: As a user evaluating the library, I want runnable examples in Python, so that I can understand expected integration patterns.

Acceptance Criteria

1. THE repository SHALL include runnable Python examples for FSM and PN under `python/examples`
2. THE example structure SHALL separate model definition, system input writer, and runtime wiring (`model.py`, `system.py`, `main.py`)

Requirement 10: Execution Model Constraints

User Story: As a platform engineer, I want clear operational constraints, so that I can integrate `rxnet` correctly in larger systems.

Acceptance Criteria

1. THE runtime SHALL be single-threaded per runtime instance unless external synchronization is provided by the integrator
2. THE library SHALL not include built-in networking, persistence, or REST APIs
3. THE library SHALL leave manual tick frequency/loop ownership to the host application
4. THE library SHALL leave I/O side effects to user-defined action callbacks
5. THE library SHALL support using one shared typed input structure across multiple model nodes in the same runtime context
6. THE Core_Runtime SHALL NOT materialise a hyperperiod activation table
7. THE cyclic, cooperative, and thread executors SHALL own their scheduling state
8. THE cyclic, cooperative, and thread executors SHALL expose schedulability analysis based on measured WCET values

9. THE runtime SHALL measure and retain the maximum critical-section access time for each task/resource pair
10. THE thread executor SHALL include shared-resource blocking in response-time analysis using lower-priority critical sections without reusing a task or resource in the same blocking combination

Design Document: rxnet — Python Implementation

Overview

This document outlines the design for the Python implementation of `rxnet`, a synchronous reactive runtime library with two model frontends (Finite State Machine and Petri Net) built on a shared phase-based core.

The core design decision is to centralize tick orchestration (`latch -> evaluate -> commit -> dump`, with deferred action dispatch between commit and dump) and let each model frontend implement only model-specific behavior on top of a generic node contract.

The design emphasizes deterministic execution, explicit input snapshotting, deferred side effects, and ergonomic dataclass-based APIs using only standard-library modules.

Architecture

High-Level Architecture

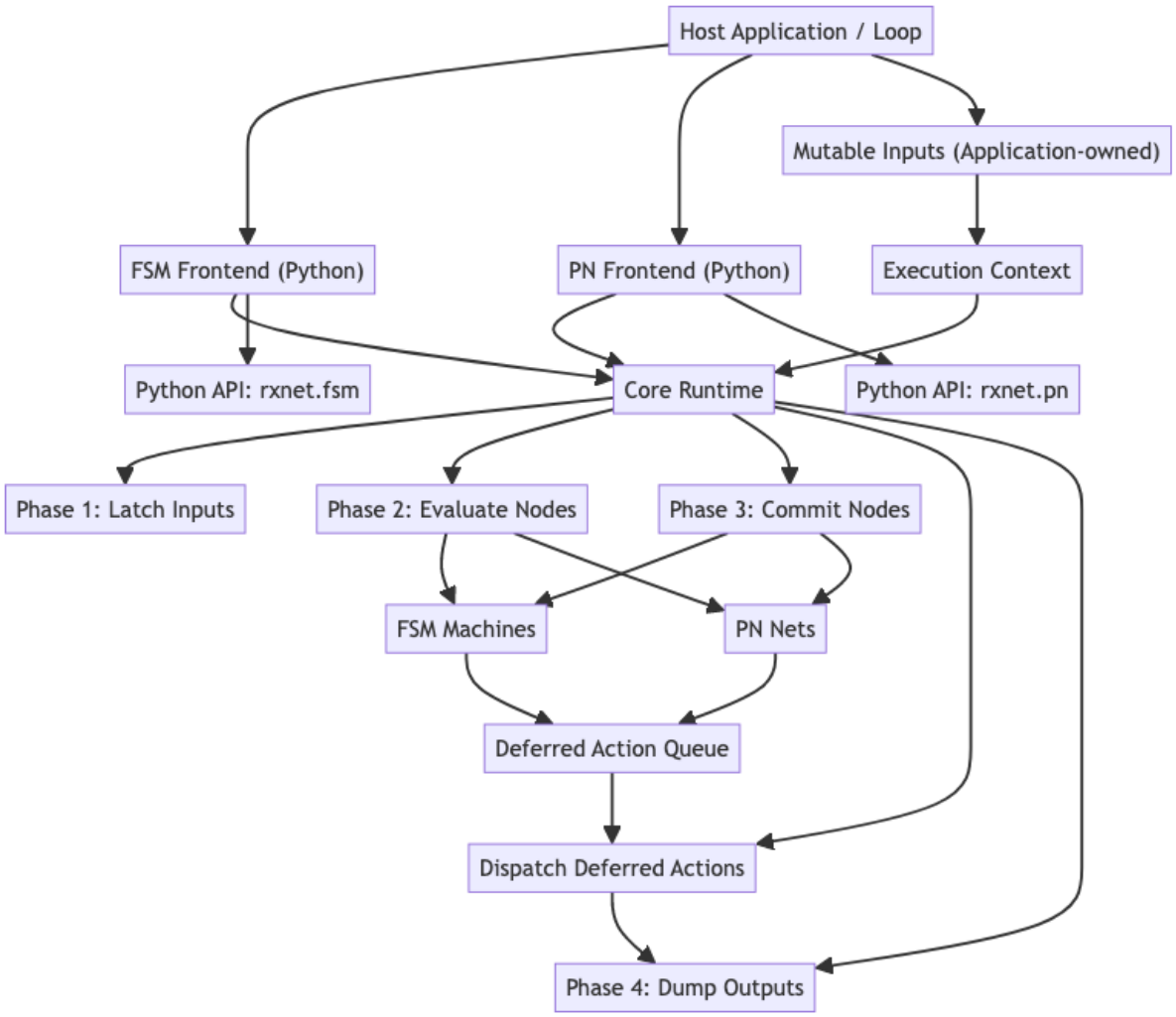


Figure 1: High-level architecture of the `rxnet` Python runtime and model frontends

Component Architecture

The system follows a layered runtime architecture:

1. **Host Integration Layer**: Application-owned inputs, tick scheduling, and output side effects
2. **Model Frontends**: FSM and PN model APIs (`rxnet.fsm`, `rxnet.pn`)

3. **Core Runtime Layer**: Generic node container and phase-ordered tick execution (`rxnet.runtime`)
4. **Execution Context Layer**: Live inputs, latched snapshot, deferred action queue
5. **Model Nodes**: Concrete node implementations (`Machine`, `Net`) with evaluate/commit logic

Components and Interfaces

Core Runtime

Responsibilities: - Maintain a list of executable nodes - Execute deterministic tick phases - Coordinate context latch and deferred actions - Store the logical activation instant (`Context.activation_us`) - Validate per-node period/deadline metadata and compute the base period - Provide minimal life-cycle surface (`add_node`, `tick`)

Key Interfaces:

```
class Node(Protocol):
    def evaluate(self, ctx: Context) -> None: ...
    def commit(self, ctx: Context) -> None: ...

class Runtime:
    def add_node(self, node: Node, period_us: int = 0, deadline_us: int
        = 0) -> None: ...
    def tick(self) -> None: ...
    def tick_nodes_at(self, node_indices: Sequence[int], activation_us:
        int) -> None: ...
```

The core runtime does not materialise a hyperperiod activation table. Multi-rate executors own scheduling state: `CyclicExecutive` builds a hyperperiod table, `CoopExecutive` tracks each node's next activation, and `ThreadExecutive` creates BSP activation groups dynamically.

Context

Responsibilities: - Hold mutable application inputs (`inputs`) - Hold immutable per-tick snapshot (`latched_inputs`) - Queue and execute deferred actions

Design Notes: - Latching is shallow copy (`copy.copy`) - The deferred-action queue grows dynamically (no fixed capacity)

Key Interfaces:

```
class Context:
    inputs: Any
```

```
    latched_inputs: Any
    activation_us: int
    def latch_inputs(self) -> None: ...
    def enqueue_deferred_action(self, fn: Any, user: Any) -> None: ...
    def run_deferred_actions(self) -> None: ...
```

FSM Frontend

Responsibilities: - Define machine transitions (`from_state`, `to_state`, `guard`, `action`) - Evaluate first valid transition in declaration order - Commit next state and enqueue optional deferred action - Read shared latched inputs from context in guards/actions

Key Interfaces:

```
@dataclass(frozen=True, slots=True)
class Transition:
    from_state: int
    to_state: int
    guard: Optional[Guard] = None
    action: Optional[Action] = None

@dataclass(slots=True)
class Machine:
    name: str
    state: int
    transitions: Sequence[Transition]
    user: Any = None
```

Petri Net Frontend

Responsibilities: - Represent places and transitions with consume/produce arcs - Evaluate transition enablement by token availability and guards - Apply transition deltas in commit phase - Enqueue transition actions as deferred side effects

Key Interfaces:

```
@dataclass(frozen=True, slots=True)
class Arc:
    place_id: int
    weight: int = 1

@dataclass(frozen=True, slots=True)
class Transition:
    consume: Sequence[Arc] = ()
    produce: Sequence[Arc] = ()
    guard: Optional[Guard] = None
```

```
    action: Optional[Action] = None

@dataclass(slots=True)
class Net:
    name: str
    places: Sequence[int]
    transitions: Sequence[Transition]
    user: Any = None
```

Package API Surface

Root package exports (`rxnet.__init__`): - `fsm` module - `pn` module - `runtime` module - FSM-oriented aliases for convenient top-level imports

Host Integration Layer

Responsibilities: - Own and mutate live inputs before each tick - Invoke tick in a periodic or event-driven loop - Reset edge-triggered inputs when required by domain logic - Implement side effects in action callbacks

Supported Patterns: - Python script loops with explicit input driver objects (per `system.py` / `main.py` example structure)

Data Models

Runtime Model

```
interface RuntimeModel {
  context: ContextModel
  nodes: NodeModel[]
  tickPhases: ['latch', 'evaluate', 'commit', 'dump']
}

interface ContextModel {
  inputs: unknown
  latchedInputs: unknown
  deferredActions: DeferredActionModel[]
}

interface DeferredActionModel {
  fn: Function
  user: unknown
}
```

FSM Model

```ts
interface FSMMachineModel {
  name: string
  state: number
  nextState: number
  transitions: FSMTransitionModel[]
  user?: unknown
}

interface FSMTransitionModel {
  fromState: number
  toState: number
  guard?: (ctx: unknown, user: unknown) => boolean
  action?: (ctx: unknown, user: unknown) => void
}
```

PN Model

```ts
interface PNNetModel {
  name: string
  places: number[]
  nextPlaces: number[]
  transitions: PNTransitionModel[]
  fireFlags: boolean[]
  user?: unknown
}

interface PNTransitionModel {
  consume: PNArcModel[]
  produce: PNArcModel[]
  guard?: (ctx: unknown, user: unknown) => boolean
  action?: (ctx: unknown, user: unknown) => void
}

interface PNArcModel {
  placeId: number
  weight: number
}
```

Correctness Properties

A property is a behavior that should hold for all valid executions. Properties connect requirements with verifiable guarantees in unit, integration, and property-based tests.

Property 1: Phase Ordering Determinism

For any tick execution, phase order should always be `latch -> evaluate -> commit -> dump`, with deferred action dispatch after commit and before dump. **Validates: Requirements 1.1, 1.2, 1.3, 1.4, 1.5, 1.6**

Property 2: Snapshot Consistency Within Tick

For any guard evaluation during one tick, observed inputs should come from `latched_inputs` and remain stable for that tick. **Validates: Requirements 2.1, 2.2, 2.3**

Property 3: Deferred Action Isolation

For any action emitted during commit, execution should occur only after all nodes have completed commit. **Validates: Requirements 3.1, 3.2**

Property 4: Deferred Queue Reset

For any completed tick, deferred queue length should be zero after running deferred actions. **Validates: Requirements 3.3**

Property 5: Runtime Node Contract Safety

For any node registered in the runtime, `evaluate` and `commit` should both be callable in every tick. **Validates: Requirements 4.1**

Property 6: FSM First-Match Transition Rule

For any FSM machine and state, transition selection should be the first declaration-order transition that matches state and guard. **Validates: Requirements 5.1, 5.2**

Property 7: FSM No-Match Stability

For any FSM tick with no valid transition, machine state should remain unchanged. **Validates: Requirements 5.3**

Property 8: FSM Deferred Action Behavior

For any matched FSM transition with action, the action should be queued for deferred execution, not run inline. **Validates: Requirements 5.5**

Property 9: PN Transition Enablement

For any PN transition, firing should require valid arcs and sufficient consume tokens. **Validates: Requirements 6.2**

Property 10: PN Guard Enforcement

For any enabled PN transition with guard, transition should fire only when guard is true. **Validates: Requirements 6.3**

Property 11: PN Commit Delta Correctness

For any set of fireable PN transitions, place deltas should match arc consume/produce sums in commit. **Validates: Requirements 6.4**

Property 12: PN Invalid Arc Exceptions

For any Python PN evaluation with invalid arc data, runtime should raise typed exceptions (`IndexError` / `ValueError`). **Validates: Requirements 7.1, 7.2**

Property 13: Python API Surface Stability

For any supported Python import path, package exports and runtime wrappers should match documented API. **Validates: Requirements 8.1, 8.2, 8.3, 8.4**

Property 14: Example Executability

For any provided Python example entrypoint, example should run and complete without runtime errors in a valid Python environment. **Validates: Requirements 9.1, 9.2**

Property 15: Host-Controlled Scheduling

For any deployment context, tick frequency and loop ownership should remain controlled by host code, not by runtime internals. **Validates: Requirements 10.3**

Error Handling

Error Categories

1. Model Validation Errors (exception based) - PN invalid `place_id` → raises `IndexError` - PN negative arc weight → raises `ValueError`

2. Integration Errors - Host passes invalid input lifetimes - Host loop omits input reset logic for edge-triggered signals

Error Handling Strategy

Fail-fast typed exceptions: - Invalid PN arc data raises typed exceptions during evaluation - User callbacks may raise and should be handled by host application policy

Host strategy: explicit loop ownership - Host catches exceptions per tick - Host applies retry/abort policy appropriate to the runtime context

Testing Strategy

Unit Tests

Focus on: - Per-function semantics in runtime/fsm/pn modules - Edge cases (empty transitions, invalid arcs) - Correct deferred action behavior

Property Tests

Framework: `hypothesis`

Configuration guidelines: - Minimum 100 runs per property - Each property references its design property number - Seeded reproducibility for failure triage

Example Property Test Structure:

```python
# Property 6: FSM First-Match Transition Rule
@given(st.lists(transition_strategy, min_size=1, max_size=20))
def test_fsm_first_match_ordering(transitions):
    machine = Machine(name="m", state=0, transitions=transitions)
    rt = Runtime(inputs=DummyInputs())
    rt.add_machine(machine)
    rt.tick()
    # Assert selected transition corresponds to first matching
       transition
```

Integration Tests

Focus on: - Host inputs mutation -> runtime tick -> state update -> deferred side effects - Multiple nodes sharing one context input struct - Python example execution paths

Code Quality and Development Standards

- Use type hints and dataclasses consistently
- Keep runtime/frontend modules dependency-light (standard library only in core)
- Preserve API compatibility for existing imports from `rxnet.__init__`

Non-Goals and Out-of-Scope

- No built-in networking or REST API layer
- No built-in persistence, serialization, or storage backends
- No built-in scheduler, thread pool, or realtime policy manager
- No automatic synchronization for concurrent access to one runtime instance

These concerns are intentionally delegated to host applications.

Implementation Plan: rxnet — Python Implementation

Overview

This implementation plan converts the `rxnet` Python design into discrete tasks for completing and validating the library. It follows an incremental approach: core runtime semantics first, model frontends second, integration examples third, and verification/quality automation last.

Tasks are marked as completed when they reflect the current repository state, and left pending where implementation, tests, or automation are still missing.

Tasks

- ☒ 1. Establish Python repository structure
  - ☒ 1.1 Create Python package layout and modules
    - Add `python/rxnet` package with `runtime.py`, `fsm.py`, `pn.py`, `__init__.py`
    - *Requirements: 8.1, 8.2, 8.3, 8.4*
  - ☒ 1.2 Add top-level README docs
    - Document phase model and basic execution usage
    - *Requirements: 9.1*
- ☒ 2. Implement shared synchronous core runtime
  - ☒ 2.1 Implement Python context with latched input snapshot
    - Use `copy.copy` snapshot semantics
    - *Requirements: 2.1, 2.2, 2.3*
  - ☒ 2.2 Implement Python deferred action queue
    - Enqueue and execute post-commit actions

    - *Requirements: 3.1, 3.2, 3.3*
  - ☒ 2.3 Implement Python generic runtime orchestration
    - Node registration and deterministic phase-ordered tick
    - *Requirements: 1.1, 1.2, 1.3, 1.4, 1.5, 4.1, 4.2*

- ☒ 3. Implement FSM and PN frontends
  - ☒ 3.1 Implement dataclass-based FSM API
    - `Machine` and `Transition` with first-match semantics
    - *Requirements: 5.1, 5.2, 5.3, 5.4, 5.5, 8.1*
  - ☒ 3.2 Implement dataclass-based PN API
    - `Net`, `Transition`, `Arc` and commit delta application
    - *Requirements: 6.1, 6.2, 6.3, 6.4, 6.5, 8.2*
  - ☒ 3.3 Implement Python validation behavior
    - Raise `IndexError` for out-of-range `place_id`
    - Raise `ValueError` for negative arc weight
    - *Requirements: 7.1, 7.2*
  - ☒ 3.4 Expose package API aliases
    - Export modules and FSM-oriented aliases in `rxnet.__init__`
    - *Requirements: 8.3, 8.4, 8.5*

- ☒ 4. Provide runnable Python examples and host integration patterns
  - ☒ 4.1 Implement Python FSM and PN examples
    - Split model/system/main integration structure
    - *Requirements: 9.1, 9.2, 10.3*
  - ☒ 4.2 Implement interactive CLI FSM examples (01-light, 02-auto, 03-blink, 04-mix)
    - Equivalent to C `examples/fsm/00-light` through `fsm/03-mix`
    - Shared `app_driver.py` (host-mock GPIO) + `cli.py` (non-blocking stdin)
    - *Requirements: 9.1, 9.2, 10.3*
  - ☒ 4.3 Implement interactive CLI PN examples (01-light, 02-auto, 03-blink, 04-mix)
    - Equivalent to C `examples/pn/00-light` through `pn/03-mix`
    - REQUEST places, AUTO_OFF_DUE signal place, TOGGLE_DUE signal place
    - *Requirements: 9.1, 9.2, 10.3*

- ☒ 5. Create baseline Python specification documents
  - ☒ 5.1 Write Python requirements specification

    - Add `docs/specs/python/requirements.md` as source of truth for Python behavior
  - ☒ 5.2 Write Python design specification
    - Add `docs/specs/python/design.md` with architecture, interfaces, and properties

- ☒ 6. Add Python unit and property-based tests
  - ☒ 6.1 Add runtime/frontend unit tests
    - 56 tests covering runtime (Context + Runtime), FSM (lifecycle, transitions, guards, actions, callbacks), and PN (lifecycle, firing, guards, actions, greedy-sequential, callbacks)
    - *Requirements: 1.5, 5.2, 6.2, 7.1, 7.2*
  - [ ]* 6.2 Add property tests for correctness properties
    - **Property 1: Phase Ordering Determinism**
    - **Property 6: FSM First-Match Transition Rule**
    - **Property 11: PN Commit Delta Correctness**
    - **Property 12: PN Invalid Arc Exceptions**
    - *Requirements: 1.1, 5.2, 6.4, 7.1, 7.2*

- ☒ 7. Add C/Python semantic parity test harness
  - ☒ 7.1 Define shared scenario corpus
    - Three scenarios: `fsm_light`, `fsm_first_match`, `pn_light` (same as C corpus)
    - *Requirements: 1.5*
  - ☒ 7.2 Implement cross-language conformance runner (Python side)
    - `python/tests/parity_runner.py` outputs identical 15-line trace to C runner
    - `tests/parity/run_parity.sh` diffs both; CI `parity` job runs the check
    - *Requirements: 1.5, 8.4*

- ☒ 8. Implement developer quality automation
  - ☒ 8.1 Add formatting/linting/type-check tools
    - `ruff` (lint + format) and `mypy` configured in `pyproject.toml` [`tool.ruff`]/[`tool.mypy`]
    - Run: `uv run --extra dev ruff check rxnet/ tests/` and `uv run --extra dev mypy rxnet/`
    - *Requirements: 8.5*
  - ☒ 8.2 Add pre-commit and CI checks

    - `.github/workflows/ci.yml`: `python-tests` job runs ruff + mypy + pytest on every PR
    - `.pre-commit-config.yaml`: ruff + ruff-format hooks for Python files
    - *Requirements: 9.1, 9.2*

- ☒ 9. Add Python packaging and release workflow
  - ☒ 9.1 Define Python packaging metadata and install flow
    - `pyproject.toml` with hatchling build backend; `dev` extras include pytest, ruff, mypy
    - Run tests: `uv run --extra dev pytest tests/ -v`
    - *Requirements: 8.3, 9.1*
  - ☒ 9.2 Document versioning and compatibility rules
    - Versioning policy: increment `version` in `pyproject.toml` on any public API change. PR checklist: requirements → design → tasks → code.
- ☒ 10. Final validation and readiness checkpoint
  - ☒ 10.1 Verify all correctness properties are covered by tests
    - 56 unit tests cover all design properties (Context, Runtime, FSM, PN semantics)
    - Parity runner covers Property 1 (phase ordering), 6 (first-match), 11 (PN delta)
  - ☒ 10.2 Verify requirements-to-implementation traceability
    - All requirements §1–§8 have implemented code + pytest coverage
    - Parity harness covers §1.5 (cross-language conformance)
  - ☒ 10.3 Establish “requirements-first” change workflow
    - PR rule in `.github/workflows/ci.yml` and documented here: update `requirements.md` → `design.md` → `tasks.md` before behavioral code changes
- ☒ 11. Reescribir README con guía real de integración
  - ☒ 11.1 Reescribir `python/README.md`
    - Añadir explicación del modelo, cuándo usar FSM vs PN, patrón básico de integración
    - Añadir sección de concurrencia: GIL, threading.Thread, asyncio (tick en corutina), ejecutivo cíclico
  - ☒ 11.2 Actualizar `README.md` raíz para incluir sección Python con getting-started

Notes

- Tasks marked with ⋆ are optional for early MVP but recommended for robustness.

- This file tracks implementation status against current repository state as of April 2026.
- Any behavior change should first update `docs/specs/python/requirements.md`, then `docs/specs/python/design.md`, then this task plan.